

PROJETO INTEGRADOR · ADS 4º PERÍODO

DOCUMENTAÇÃO TÉCNICA

E-Commerce de Camisetas

Entrega N2 — Documento Unificado

Funcionalidade	Arquitetura	Qualidade & Testes	UI em Tempo Real
Stack	Java 21 · Spring Boot 3.x · Apache Kafka · React + Vite · Docker		
Curso	Análise e Desenvolvimento de Sistemas — 4º Período		
Instituição	PUC Goiás · Semestre 2026/1		
Autores	Victor Hugo · Josue Felix · Guilherme Bastos		

Sumário

1.	Visão Geral e Contexto N2	3
2.	Fluxo Completo — Cadastro → Pedido → Mensageria → Tela	4
3.	Arquitetura Implementada — Diagrama C4 N2	6
4.	Design Patterns Aplicados	8
5.	Qualidade de Código — Clean Architecture e Estrutura	10
6.	Testes Unitários e Cobertura (JaCoCo)	11
7.	Interface do Usuário — Tempo Real com WebSocket	13
8.	Aderência aos Critérios de Avaliação N2	14

1. Visão Geral e Contexto N2

A entrega N2 marca a transição do plano arquitetural (N1) para o software funcionando. O sistema TeeStore é uma plataforma de e-commerce de camisetas que implementa o fluxo completo exigido pelo documento norteador: **Cadastro** → **Pedido** → **Processamento na Fila (Mensageria)** → **Atualização na Tela (Web)**. O diferencial técnico central é o desacoplamento assíncrono via Apache Kafka — o Backend não espera o processamento logístico para responder ao cliente.

Evolução da Arquitetura: N1 → N2

A N1 projetou 5 microsserviços independentes com API Gateway. Na N2, o que foi entregue é uma arquitetura mais pragmática e coesa: um Backend modular monolítico que concentra toda a lógica de negócio, complementado por dois serviços especializados e autônomos.

Aspecto	N1 (Planejado)	N2 (Implementado)
Backend	5 microsserviços + API Gateway	1 Backend modular (:8080) + 2 serviços especializados
Autenticação	Não detalhada	JWT stateless + Refresh Token (Redis) + Blacklist
Carrinho	Não detalhada	Redis com TTL (sem banco relacional)
Logística	order-service (:8082)	Logistics Service (:8082) — Virtual Threads + Idempotência
Notificação	notification-service (:8083)	Notification Service (:8083) — WebSocket STOMP stateless
Tópicos Kafka	order.created, order.updated	order.created, order.status.updated, notification.send
Testes	Planejados com Testcontainers	58 testes unitários (JUnit+Mockito) + JaCoCo + CI/CD
Documentação API	Não prevista	Swagger/OpenAPI integrado (:8080/swagger-ui.html)
Infraestrutura	1 Kafka broker	Cluster Kafka com 3 brokers (29092, 29093, 29094)

Stack Tecnológico Implementado

Camada	Tecnologia	Detalhe
Backend	Java 21 + Spring Boot 3.x	Web, Data JPA, Kafka, WebSocket, Security, Actuator
Mensageria	Apache Kafka 3.5	Cluster 3 brokers, acks=all, idempotência, auto.offset=earliest
Banco de Dados	PostgreSQL 16	backend-db (:5432) + logistics-db (:5435) — bancos separados
Cache / Sessão	Redis 7	Carrinho (TTL), Refresh Tokens, Token Blacklist
Frontend	React 18 + Vite 5	STOMP.js + SockJS, CSS Modules, Axios com interceptor JWT

Testes	JUnit 5 + Mockito + JaCoCo	58 testes unitários, relatório HTML, GitHub Actions CI/CD
Documentação API	Swagger / OpenAPI 3	Acessível em /swagger-ui.html com todos os endpoints
Infraestrutura	Docker Compose	Kafka cluster, ZooKeeper, 2x PostgreSQL, Redis, serviços
Threads	Java 21 Virtual Threads	spring.threads.virtual.enabled=true em todos os serviços

2. Fluxo Completo — Cadastro → Pedido → Mensageria → Tela

Este fluxo é o núcleo do critério de **Implementação Técnica (0,8 pts)**: o software deve funcionar ponta a ponta (0,4 pts) e a mensageria deve estar desacoplada corretamente (0,4 pts). Cada etapa abaixo corresponde a um componente real em produção.

2.1 Visão Macro — Pipeline de Eventos

1	CADASTRO	Usuário cria conta via POST /auth/register. Backend valida, persiste no PostgreSQL, gera JWT access token + refresh token armazenado no Redis.
2	LOGIN	POST /auth/login → AuthenticationManager valida credenciais, retorna JWT (1h) + refresh token (7 dias). Frontend armazena no localStorage.
3	NAVEGAÇÃO & CARRINHO	Usuário adiciona produtos ao carrinho. CartService serializa os itens em JSON e persiste no Redis com TTL. Zero acesso ao PostgreSQL.
4	CONFIRMAÇÃO DO PEDIDO	POST /orders → OrderService cria Order no PostgreSQL (status PROCESSING), publica evento OrderCreatedEvent no tópico Kafka order.created. Backend retorna 201 imediatamente — sem esperar logística.
5	KAFKA: order.created	Broker Kafka retém a mensagem no tópico order.created (3 brokers, acks=all). Consumer group logistics-group recebe a mensagem assim que disponível.
6	LOGISTICS SERVICE	OrderEventConsumer consome order.created. LogisticsProcessor (Virtual Thread): persiste LogisticsOrder (PROCESSING) → aguarda 8s → publica order.status.updated (SHIPPED) → aguarda 15s → publica order.status.updated (DELIVERED). Idempotência: existsById() evita reprocessamento.
7	KAFKA: order.status.updated	Tópico order.status.updated recebe os eventos SHIPPED e DELIVERED do Logistics. Dois consumidores independentes: notification-group e backend-group.
8	NOTIFICATION SERVICE	StatusEventConsumer consome order.status.updated. Monta WebSocketPayload (orderId, type, message, trackingCode, sentAt) e envia via SimpMessagingTemplate para /topic/notifications/{userId}.
9	ATUALIZAÇÃO NA TELA	Frontend (useNotifications.js) recebe o frame WebSocket STOMP. Exibe toast popup com a mensagem. Dispara CustomEvent teestore-order-update → Orders.jsx re-faz GET /orders/my automaticamente sem recarregar a página.

2.2 Tópicos Kafka — Produtores e Consumidores

Tópico	Produtor	Consumidor(es)	Partições	Quando
order.created	Backend Service (OrderService)	Logistics Service (logistics-group)	3	Ao finalizar pedido no checkout

order.status.updated	Logistics Service (OrderStatusProducer)	Notification Service (notification-group) Backend (backend-status-group)	3	SHIPPED (8s) e DELIVERED (15s) após criação
notification.send	Backend Service (OrderEventProducer)	Notification Service (notification-group)	3	Admin atualiza status manualmente

2.3 Configuração Kafka (application.yml)

```
spring.kafka.bootstrap-servers: localhost:29092, localhost:29093, localhost:29094
spring.kafka.producer.acks: all
spring.kafka.producer.enable-idempotence: true
spring.kafka.consumer.auto-offset-reset: earliest
spring.kafka.consumer.group-id: logistics-group # ou notification-group
spring.threads.virtual.enabled: true # Java 21 Virtual Threads
```

Desacoplamento verificado: O Backend Service publica o evento em order.created e retorna HTTP 201 ao cliente imediatamente. O Logistics Service consome de forma independente — se estiver offline, a mensagem fica retida no broker e será processada quando voltar. O Backend não conhece a existência do Logistics Service.

3. Arquitetura Implementada — Diagrama C4 N2

3.1 Nível Contexto (C1)

O sistema TeeStore é uma plataforma web de e-commerce. Os atores externos são o Cliente (comprador), o Administrador (gestor da loja) e o Apache Kafka (broker de mensagens, infraestrutura externa ao domínio de negócio).

Ator / Sistema	Tipo	Descrição
Cliente	Pessoa	Navega no catálogo, adiciona ao carrinho, finaliza pedido e acompanha entrega em tempo real via WebSocket no browser.
Administrador	Pessoa	Cadastra e edita produtos (com upload de imagem), visualiza todos os pedidos, atualiza status manualmente e monitora dashboard de métricas.
Apache Kafka	Sistema Externo	Broker de mensagens. Recebe eventos do Backend (order.created, notification.send) e os distribui para Logistics e Notification Service.

3.2 Nível Contêineres (C2) — O que foi implementado

Frontend SPA :5173	<i>React 18 + Vite + SockJS/STOMP</i>	Interface completa: vitrine, produto, carrinho, checkout, pedidos, perfil, painel admin. Usa CSS Modules, Axios com interceptor de JWT e SockJS para WebSocket em tempo real.
Backend Service :8080	<i>Spring Boot 3.x + JPA + Redis</i>	API REST principal. Auth (JWT+Redis), produtos (CRUD+S3-like upload), carrinho (Redis), pedidos, admin, dashboard, notificações DB. Swagger em /swagger-ui.html. Produz: order.created, notification.send.
Logistics Service :8082	<i>Spring Boot 3.x + JPA + Kafka</i>	Consume order.created. Simula ciclo logístico via Virtual Thread: PROCESSING → SHIPPED (8s) → DELIVERED (15s). Persiste em logistics-db. Idempotência por orderId. Produz: order.status.updated.
Notification Service :8083	<i>Spring Boot 3.x + WebSocket</i>	Consume order.status.updated e notification.send. Entrega mensagens ao browser via STOMP em /topic/notifications/{userId}. Completamente stateless — sem banco de dados.
Apache Kafka 29092-29094	<i>Kafka 3.5 + ZooKeeper</i>	Cluster de 3 brokers para alta disponibilidade. Tópicos: order.created, order.status.updated, notification.send. Consumer groups: logistics-group, notification-group.
PostgreSQL (Backend) :5432	<i>PostgreSQL 16</i>	Banco principal: users, products, orders, order_items, notifications. Gerenciado por Flyway (migrations versionadas).
PostgreSQL (Logistics) :5435	<i>PostgreSQL 16</i>	Banco dedicado ao Logistics Service: logistics_orders com ciclo de vida logístico. Flyway: V1 (criação), V2 (compatibilidade Hibernate).

Redis :6379	<i>Redis 7</i>	Três usos: carrinho (hash por userId), refresh tokens (refresh:{token} → userId), blacklist de tokens revogados.
-----------------------	----------------	--

4. Design Patterns Aplicados

O critério de **Qualidade de Código (0,5 pts)** exige aplicação de padrões definidos em Design de Software: Injeção de Dependência, Strategy, Factory, etc. Os padrões abaixo foram aplicados para resolver problemas reais do projeto — não como ornamento acadêmico.

Injeção de Dependência (DI) Onde: Todos os serviços Spring Boot	
Todas as dependências são injetadas via construtor, nunca via campo (@Autowired). O LogisticsProcessor recebe LogisticsOrderRepository e OrderStatusProducer; o StatusEventConsumer recebe SimpMessagingTemplate. Classes completamente testáveis sem subir o contexto Spring.	<pre>public LogisticsProcessor(LogisticsOrderRepository repository, OrderStatusProducer producer) { this.repository = repository; this.producer = producer; }</pre>
Strategy Onde: StatusEventConsumer (Notification)	
A construção da mensagem de notificação aplica Strategy via switch expression do Java 21. Cada status de pedido tem seu próprio comportamento encapsulado. Adicionar um novo status (ex: RETURNED) não altera o código existente — apenas adiciona um novo case.	<pre>private String buildMessage(String status, String orderId) { return switch (status) { case "SHIPPED" -> "Pedido #" + id + " enviado!"; case "DELIVERED" -> "Pedido #" + id + " entregue!"; default -> "Status do pedido #" + id + " atualizado para " + status; }; }</pre>
Observer (via Kafka + STOMP) Onde: Backend → Logistics → Notification → Frontend	
O padrão Observer é implementado em duas camadas. Na camada de backend: o Backend publica no Kafka sem conhecer os consumidores (Logistics e Notification são observadores desacoplados). Na camada frontend: o frontend se inscreve em /topic/notifications/{userId} e recebe push sem polling — zero acoplamento direto.	<pre>// Notification Service publica para o browser: messagingTemplate.convertAndSend("/topic/notifications/" + userId, payload); // Frontend (useNotifications.js): client.subscribe("/topic/notifications/" + id, msg => dispatch(parse(msg.body)));</pre>
Template Method Onde: LogisticsProcessor	
O método processOrder define o esqueleto fixo do fluxo logístico: receber → salvar como PROCESSING → aguardar shippingDelay → publicar SHIPPED → aguardar deliveryDelay → publicar DELIVERED. Os delays são configurados via @Value, externalizáveis por .env sem recompilação.	<pre>@Value("\${app.logistics.shipping-delay-ms:8000}") private long shippingDelayMs; // No processOrder(): Thread.sleep(shippingDelayMs); producer.publishStatus(orderId, userId, "SHIPPED");</pre>

Repository		Onde: Todos os serviços com JPA
Spring Data JPA isola a camada de acesso a dados. Cada serviço tem seu próprio repositório (OrderRepository, LogisticsOrderRepository, ProductRepository, etc.) que é mockado nos testes unitários via Mockito. Nenhum service acessa EntityManager diretamente.	<pre>public interface LogisticsOrderRepository extends JpaRepository { boolean existsById(UUID orderId); Optional findById(UUID orderId); }</pre>	
Idempotência		Onde: LogisticsProcessor
Antes de processar qualquer evento Kafka, o Logistics Service verifica se o orderId já existe no banco. Essa guarda garante que reentregas do broker (at-least-once delivery) não gerem pedidos duplicados — padrão crítico em sistemas de mensageria.	<pre>if (repository.existsById(event.orderId())) { log.warn("Pedido {} já processado.", event.orderId()); return; // ignora duplicata }</pre>	

5. Qualidade de Código — Clean Architecture e Estrutura

Além dos padrões de projeto, o critério de Qualidade de Código avalia Arquitetura Limpa e legibilidade. Os três serviços seguem uma estrutura de pacotes uniforme baseada na separação de responsabilidades.

5.1 Estrutura de Pacotes — Padrão Adotado

Pacote	Responsabilidade	Regra Clean Architecture
controller/	@RestController · DTOs de entrada/saída	Borda externa — conhece HTTP
service/	@Service · Lógica de negócio · Padrões	Core — só depende de domain e repository
repository/	JpaRepository · acesso a dados	Infraestrutura — mockado nos testes
messaging/	KafkaTemplate (Producer) · @KafkaListener (Consumer)	Infraestrutura — não toca domain
domain/entity/	Entidades JPA · zero Spring no domínio	Core puro — sem import do framework
domain/enums/	Enums de estado (OrderStatus, Role, Size...)	Core puro
dto/	Java Records (Request / Response / Event)	Imutáveis · thread-safe · zero boilerplate
config/	Beans de configuração (Kafka, Security, WebSocket)	Infraestrutura
exception/	Exceções de domínio + GlobalExceptionHandler	Tratamento centralizado (RFC 7807)

5.2 Java 21 — Features Modernas Utilizadas

Feature	Onde	Benefício
Virtual Threads (Project Loom)	application.yml: spring.threads.virtual.enabled=true (todos os 3 serviços)	Substitui thread pool por threads leves gerenciadas pelo JVM. O Logistics Service processa centenas de pedidos em paralelo sem esgotar threads de SO.
Records (DTOs imutáveis)	OrderCreatedEvent, OrderStatusEvent, WebSocketPayload, CartItemData	Zero boilerplate: construtor, getters, equals/hashCode e toString gerados pelo compilador. Thread-safe por natureza.
Switch Expression (Java 14+)	StatusEventConsumer.buildMessage()	Implementa o padrão Strategy de forma concisa. O compilador alerta se algum case for esquecido.
Text Blocks	Queries SQL em @Query dos repositories	Strings multilinhas legíveis sem concatenação.

6. Testes Unitários e Cobertura (JaCoCo)

O critério de **Qualidade e Testes (0,4 pts)** exige existência de testes unitários rodando. Os três serviços possuem suítes JUnit 5 + Mockito com relatório de cobertura JaCoCo gerado automaticamente via CI/CD.

6.1 Backend Service — Suíte de Testes (58 casos)

Classe de Teste	Casos	O que é testado
AuthServiceTest	8	register/login com sucesso, email duplicado, refresh token (rotação e expiração), logout com e sem refresh token
ProductServiceTest	5	listagem paginada, busca por ID, criação, atualização e exclusão de produto
CartServiceTest	7	getCart vazio, addItem (novo e acumulação), updateItem, removeItem, clearCart, produto não encontrado
UserServiceTest	4	getProfile, updateProfile, alteração de senha com validação, usuário não encontrado
OrderServiceTest	6	createOrder (fluxo completo), listagem por usuário, toResponse mapping, cálculo de total
OrderAdminServiceTest	7	getAllOrders (filtro/sem filtro), getOrderById, updateOrderStatus (SHIPPED e DELIVERED), publicação Kafka, não encontrado
NotificationQueryServiceTest	4	listagem por usuário, marcar como lida, contagem de não lidas
DashboardServiceTest	5	métricas: total pedidos, receita, produtos ativos, pedidos por status, pedidos recentes
FileUploadServiceTest	4	upload de imagem, validação de tipo, tamanho máximo, exclusão de arquivo

6.2 Logistics Service e Notification Service

Serviço	Classe de Teste	Casos	Cenários cobertos
Logistics	LogisticsProcessorTest	5	Ciclo completo PROCESSING→SHIPPED→DELIVERED, idempotência (orderId duplicado ignorado), geração de tracking code, publicação dos dois eventos Kafka na ordem correta
Notification	StatusEventConsumerTest	4	Consumo de SHIPPED e DELIVERED, construção correta do WebSocketPayload (type, message, trackingCode), entrega via SimpMessagingTemplate, tratamento de JSON inválido
Notification	NotificationEventConsumerTest	3	Consumo do tópico notification.send, mapeamento correto do payload para o userId destino, entrega WebSocket para o canal correto

6.3 Como Acessar os Relatórios de Cobertura

Dois tipos de relatório são gerados automaticamente por mvn verify:

Relatório	Localização Local	Via GitHub Actions
JaCoCo (cobertura de código)	target/site/jacoco/index.html	Actions → Artifacts → jacoco-coverage-report → index.html
Surefire (execução dos testes)	target/site/surefire-report.html	Actions → Artifacts → surefire-test-report → surefire-report.html

6.4 Estratégia de Testes — @Disabled nas Classes de Integração

Os testes `@SpringBootTest` nas classes `ApplicationTests` são anotados com `@Disabled` porque requerem infraestrutura completa (PostgreSQL + Redis + Kafka). Os testes unitários (JUnit + Mockito, sem Spring context) rodam de forma isolada e rápida em qualquer ambiente — incluindo o CI/CD do GitHub Actions sem Docker.

Comando para rodar localmente: mvn verify

Gera ambos os relatórios em target/site/. Para abrir: start target/site/jacoco/index.html (Windows) ou open target/site/jacoco/index.html (Mac).

7. Interface do Usuário — Tempo Real com WebSocket

O critério de **Inovação e UI Final (0,3 pts)** avalia se a interface é fiel ao protótipo e agradável. O sistema usa CSS Modules com design coeso, além de notificações em tempo real sem necessidade de refresh da página.

7.1 Páginas e Rotas Implementadas

Rota	Página	Proteção	Funcionalidade
/	Home	Pública	Vitrine de produtos com busca por nome, filtro por tamanho, favoritos (localStorage)
/login	Login	Pública	Formulário de autenticação, link para cadastro
/register	Cadastro	Pública	Formulário de registro com validação de campos
/checkout	Checkout	Autenticado	Resumo do carrinho, confirmação do pedido, esvazia carrinho pós-confirmação
/orders	Meus Pedidos	Autenticado	Lista de pedidos com status atualizado em tempo real via WebSocket
/profile	Perfil	Autenticado	Edição de dados pessoais e alteração de senha
/admin	Dashboard Admin	Admin	Métricas (total pedidos, receita, produtos), pedidos recentes
/admin/products	Produtos Admin	Admin	CRUD completo com upload de imagem
/admin/orders	Pedidos Admin	Admin	Listagem paginada, filtro por status, atualização manual de status

7.2 Notificações em Tempo Real — Implementação WebSocket

O hook `useNotifications.js` conecta ao Notification Service via SockJS/STOMP ao montar o componente. Ao receber um frame WebSocket, além de atualizar o estado local de notificações, dispara um `CustomEvent` (`teestore-order-update`) que o componente `Orders.jsx` escuta para re-buscar a lista de pedidos automaticamente — sem polling, sem refresh.

Etapa	Componente	Ação
1. Conexão	<code>useNotifications.js</code> (hook)	SockJS conecta em <code>ws://localhost:8083/ws</code> ao montar a página
2. Subscrição	<code>useNotifications.js</code>	<code>client.subscribe('/topic/notifications/{userId}', handler)</code>
3. Recepção	<code>useNotifications.js</code>	Parseia JSON → adiciona à lista local → mostra badge no Header
4. CustomEvent	<code>useNotifications.js</code>	<code>window.dispatchEvent(new CustomEvent('teestore-order-update'))</code>
5. Re-fetch	<code>Orders.jsx</code>	<code>useEffect</code> ouve 'teestore-order-update' → re-faz GET <code>/orders/my</code>
6. Exibição	<code>Orders.jsx</code> + <code>NotificationsDrawer</code>	Lista de pedidos atualizada + drawer de notificações aberto

8. Aderência aos Critérios de Avaliação N2

A tabela abaixo mapeia cada critério da rubrica N2 (2,0 pts) para os artefatos concretos entregues pelo projeto TeeStore.

Critério	Pts	Como este projeto atende	Artefato / Evidência
Implementação Técnica — Software funciona	0,4	Fluxo completo funcionando: cadastro, login, carrinho, pedido, logística simulada, notificação WebSocket.	Backend :8080 · Logistics :8082 · Notification :8083 · Frontend :5173 · Docker Compose
Implementação Técnica — Mensageria desacoplada	0,4	Backend publica em order.created e retorna 201 sem esperar o Logistics. Logistics consome de forma independente e publica order.status.updated. Notification consome e entrega via WebSocket — três serviços autônomos.	Tópicos: order.created, order.status.updated, notification.send · 3 brokers Kafka · Consumer groups isolados
Qualidade de Código — Design Patterns e Arq. Limpa	0,5	DI (construtor), Strategy (switch expression), Observer (Kafka+STOMP), Template Method (ciclo logístico), Repository (JPA), Idempotência. Pacotes domain/ sem importar Spring. DTOs como Java Records.	Seção 4 deste documento · READMEs dos repositórios GitHub
Qualidade e Testes — Unit Tests rodando	0,4	58 testes unitários no Backend + 5 no Logistics + 7 no Notification. JaCoCo gera relatório HTML de cobertura. GitHub Actions executa em cada push automaticamente.	mvn verify em cada serviço · target/site/jacoco/index.html · .github/workflows/ci.yml
Inovação e UI Final — Interface fiel e agradável	0,3	Vitrine com busca/filtro, carrinho lateral (drawer), checkout, pedidos em tempo real, perfil, painel admin completo. CSS Modules com design coeso, fontes Geist, paleta de cores consistente.	Projeto-Integrador-Frontend · React 18 + Vite + CSS Modules

Nota sobre o documento norteador: O projeto visa simular um ambiente real de desenvolvimento de software complexo. A mensageria/streams é o diferencial técnico do semestre — o TeeStore implementa Kafka com cluster de 3 brokers, 3 tópicos distintos, consumer groups isolados e entrega em tempo real via WebSocket STOMP, superando o requisito mínimo de desacoplamento.